Tarefa 20: Equação do Calor 2D com Offload OpenMP para GPU

Israel Soares de Castro Filho
Matrícula: 20250070780

1 Introdução

O objetivo da tarefa é executar na GPU o exemplo de transferência de calor (`heat.c`) do tutorial de programação de GPUs com OpenMP, usando as diretivas de movimentação de dados para que a cópia entre CPU e GPU não domine a execução e a GPU não fique ociosa.

O programa resolve a equação do calor em uma grade $n \times n$ com um estêncil de 5 pontos, avançando no tempo passo a passo. O resultado é validado contra a solução exata conhecida (erro L2).

2 Metodologia

Ambiente. O código foi compilado com GCC, com OpenMP e offload para NVIDIA (`nvptx`), e executado em uma GPU NVIDIA RTX 4070 Super local, devido à fila de tarefas do NPAD. O perfilamento foi feito com o `nsys`, rastreando as chamadas CUDA.

Estratégia de dados. O ponto principal é manter os dois vetores (`u` e `u_tmp`) na GPU durante todo o laço de tempo:

```
#pragma omp target enter data map(to: u[0:n*n], u_tmp[0:n*n])
for (int t = 0; t < nsteps; ++t) {
    solve(n, alpha, dx, dt, u, u_tmp); // kernel na GPU
    tmp = u; u = u_tmp; u_tmp = tmp;  // troca de ponteiros
}
#pragma omp target exit data map(from: u[0:n*n])
```

Código 1: Diretivas de movimentação de dados

Assim, os dados vão para a GPU uma vez, os 100 passos rodam sem nenhuma cópia intermediária (a troca de ponteiros é feita no *host*, sem copiar dados) e só o resultado final volta para a CPU. O kernel usa `#pragma omp target` com `#pragma omp loop collapse(2)`.

Experimentos. Foram testadas grades com $n \in \{500, 1000, 2000, 4000, 8000\}$, 100 passos de tempo e 3 repetições por tamanho. O programa mede o tempo com o relógio do OpenMP e grava os resultados em um arquivo CSV. Cada perfil do `nsys` cobre as 3 repetições (300 chamadas do kernel), e os tempos do CSV foram medidos nessas mesmas execuções.

3 Resultados

A Tabela 1 mostra as médias das 3 repetições. O tempo de *solve* é o laço de tempo na GPU (incluindo as cópias), e o tempo total inclui também a inicialização e o cálculo do erro, feitos na CPU.

Tabela 1: Tempos médios (3 repetições, 100 passos).

n	Solve (s)	Total (s)	Fator vs. n anterior	Erro L2
500	0,0445	0,0490	–	$8{,}10 \times 10^{-10}$
1000	0,0906	0,1107	2,04	$4{,}06 \times 10^{-10}$
2000	0,2963	0,3764	3,27	$2{,}02 \times 10^{-10}$
4000	1,3071	1,6457	4,41	$9{,}46 \times 10^{-11}$
8000	5,3722	6,7985	4,11	$5{,}14 \times 10^{-11}$

A Tabela 2 resume o perfil do `nsys`. O kernel é o tempo médio de uma chamada. Ida é o volume copiado da CPU para a GPU e volta é o da GPU para a CPU, ambos por execução. A última coluna é o tempo das cópias (ida mais volta) em relação ao tempo de *solve*.

Tabela 2: Resumo do perfil (`nsys`).

n	Kernel (ms)	Kernel/Solve (%)	Ida (MB)	Volta (MB)	Cópias/Solve (%)
500	0,398	89,5	4	2	0,8
1000	0,784	86,5	16	8	1,7
2000	2,785	94,0	64	32	2,0
4000	12,75	97,6	256	128	1,7
8000	52,70	98,1	1024	512	1,7

4 Análise

Movimentação de dados. Com `enter data` e `exit data`, os vetores são copiados uma única vez por execução. As cópias ficam em torno de 2% do tempo de *solve* em todos os tamanhos, e o kernel ocupa de 87% a 98% dele. Ou seja, a GPU passa quase todo o tempo calculando, e não esperando dados. Sem essas diretivas, cada passo copiaria os vetores nos dois sentidos, o que daria, por estimativa (não medida), mais de 100 GB de tráfego para $n = 8000$, contra cerca de 1,5 GB medidos aqui.

Escalabilidade. Para as grades menores, o tempo cresce menos que 4 vezes ao dobrar n (fatores de 2,04 e 3,27), pois a GPU ainda não está totalmente ocupada e os custos fixos (lançamento do kernel, alocação) pesam. Para $n \geq 4000$ o fator se aproxima de 4, ou seja, o tempo passa a crescer com o número de pontos (n^2).

Tempo na CPU. A diferença entre o tempo total e o de *solve* vem do trabalho feito na CPU (valor inicial e erro L2) e cresce com n^2: chega a 1,43 s em $n = 8000$, cerca de 21% do tempo total.

Correção. O erro L2 diminui à medida que a grade refina e é idêntico entre as repetições, o que indica que o resultado na GPU está correto.

5 Conclusão

As diretivas `target enter data` e `target exit data` mantêm os dados na GPU durante todo o laço de tempo. Com isso, as cópias ficaram em cerca de 2% do tempo de *solve* e o kernel ocupa quase todo o restante, sem GPU ociosa por falta de dados. Como a movimentação de dados deixou de ser o problema, o próximo ganho de desempenho está na paralelização do próprio kernel.

A Código-Fonte

```c
#include <stdlib.h>
#include <stdio.h>
#include <math.h>

#include <omp.h>

// Key constants used in this program
#define PI acos(-1.0) // Pi
#define LINE "--------------------\n" // A line for fancy output

// Arquivo CSV de saída com os resultados de todas as variações
#define ARQUIVO_CSV "resultados_tarefa20.csv"

// Function definitions
void initial_value(const int n, const double dx, const double length, double *
    restrict u);
void zero(const int n, double * restrict u);
void solve(const int n, const double alpha, const double dx, const double dt, const
    double * restrict u, double * restrict u_tmp);
double solution(const double t, const double x, const double y, const double alpha,
    const double length);
double l2norm(const int n, const double * restrict u, const int nsteps, const double
    dt, const double alpha, const double dx, const double length);
void executa(const int n, const int nsteps, double *solve_time, double *total_time,
    double *norm);

// Main function
// Sem argumentos: roda todas as variações definidas abaixo e salva no CSV.
// Com 2 argumentos (n nsteps): roda apenas essa configuração (como no original).
int main(int argc, char *argv[]) {

  // ---------- Variações a executar ----------
  int tamanhos[] = {500, 1000, 2000, 4000, 8000}; // grade n x n
  int ntam = 5;
  int passos[] = {100};                        // passos de tempo
  int npassos = 1;
  int reps = 3;                                // repetições por configuração

  // Check for the correct number of arguments
  if (argc == 3) {

    // Set problem size from first argument
    int n = atoi(argv[1]);
    if (n < 0) {
      fprintf(stderr, "Error: n must be positive\n");
      exit(EXIT_FAILURE);
    }

    // Set number of timesteps from second argument
    int nsteps = atoi(argv[2]);
    if (nsteps < 0) {
      fprintf(stderr, "Error: nsteps must be positive\n");
      exit(EXIT_FAILURE);
    }

    tamanhos[0] = n;
```

```
52      ntam = 1;
53      passos[0] = nsteps;
54      npassos = 1;
55    }
56    int acrescentar = 0;
57    if (argc == 3) {
58      FILE *existente = fopen(ARQUIVO_CSV, "r");
59      if (existente != NULL) {
60        acrescentar = 1;
61        fclose(existente);
62      }
63    }
64
65    FILE *csv = fopen(ARQUIVO_CSV, acrescentar ? "a" : "w");
66    if (csv == NULL) {
67      fprintf(stderr, "Error: nao foi possivel criar %s\n", ARQUIVO_CSV);
68      exit(EXIT_FAILURE);
69    }
70    if (!acrescentar)
71      fprintf(csv, "n,nsteps,rep,tempo_solve_s,tempo_total_s,erro_l2\n");
72
73    // Aquecimento
74    int aquece = 0;
75    #pragma omp target map(tofrom: aquece)
76    {
77      aquece += 1;
78    }
79
80    for (int a = 0; a < ntam; ++a) {
81      for (int b = 0; b < npassos; ++b) {
82        for (int rep = 1; rep <= reps; ++rep) {
83
84          printf("=== n=%d | nsteps=%d | rep=%d ===\n", tamanhos[a], passos[b], rep);
85
86          double solve_time, total_time, norm;
87          executa(tamanhos[a], passos[b], &solve_time, &total_time, &norm);
88
89          fprintf(csv, "%d,%d,%d,%lf,%lf,%E\n",
90                  tamanhos[a], passos[b], rep, solve_time, total_time, norm);
91          fflush(csv);
92        }
93      }
94    }
95
96    fclose(csv);
97    printf("Resultados salvos em %s\n", ARQUIVO_CSV);
98
99    return 0;
100 }
101
102
103 // Uma execução completa do problema (corpo do main original)
104 void executa(const int n, const int nsteps, double *solve_time, double *total_time,
        double *norm_out) {
105
106   // Start the total program runtime timer
107   double start = omp_get_wtime();
108
```

```
109   //
110   // Set problem definition
111   //
112   double alpha = 0.1;        // heat equation coefficient
113   double length = 1000.0;    // physical size of domain: length x length square
114   double dx = length / (n+1); // physical size of each cell (+1 as don't simulate
          boundaries as they are given)
115   double dt = 0.5 / nsteps; // time interval (total time of 0.5s)
116
117
118   // Stability requires that dt/(dx^2) <= 0.5,
119   double r = alpha * dt / (dx * dx);
120
121   // Print message detailing runtime configuration
122   printf("\n");
123   printf(" MMS heat equation\n\n");
124   printf(LINE);
125   printf("Problem input\n\n");
126   printf(" Grid size: %d x %d\n", n, n);
127   printf(" Cell width: %E\n", dx);
128   printf(" Grid length: %lf x %lf\n", length, length);
129   printf("\n");
130   printf(" Alpha: %E\n", alpha);
131   printf("\n");
132   printf(" Steps: %d\n", nsteps);
133   printf(" Total time: %E\n", dt*(double)nsteps);
134   printf(" Time step: %E\n", dt);
135   printf(LINE);
136
137   // Stability check
138   printf("Stability\n\n");
139   printf(" r value: %lf\n", r);
140   if (r > 0.5)
141     printf(" Warning: unstable\n");
142   printf(LINE);
143
144
145   // Allocate two nxn grids
146   double *u    = malloc(sizeof(double)*n*n);
147   double *u_tmp = malloc(sizeof(double)*n*n);
148   double *tmp;
149
150   // Set the initial value of the grid under the MMS scheme
151   initial_value(n, dx, length, u);
152   zero(n, u_tmp);
153
154   // Start the solve timer
155   double tic = omp_get_wtime();
156   #pragma omp target enter data map(to: u[0:n*n], u_tmp[0:n*n])
157   for (int t = 0; t < nsteps; ++t) {
158     solve(n, alpha, dx, dt, u, u_tmp);
159
160     // Pointer swap
161     tmp = u;
162     u = u_tmp;
163     u_tmp = tmp;
164   }
165   // u_tmp também éliberado na GPU: como agora há várias execuções no mesmo
```

```
  // processo, o mapeamento antigo não pode ficar para trás.
  #pragma omp target exit data map(from: u[0:n*n]) map(release: u_tmp[0:n*n])
  // Stop solve timer
  double toc = omp_get_wtime();

  //
  // Check the L2-norm of the computed solution
  // against the *known* solution from the MMS scheme
  //
  double norm = l2norm(n, u, nsteps, dt, alpha, dx, length);

  // Stop total timer
  double stop = omp_get_wtime();

  // Print results
  printf("Results\n\n");
  printf("Error (L2norm): %E\n", norm);
  printf("Solve time (s): %lf\n", toc-tic);
  printf("Total time (s): %lf\n", stop-start);
  printf(LINE);

  *solve_time = toc - tic;
  *total_time = stop - start;
  *norm_out = norm;

  // Free the memory
  free(u);
  free(u_tmp);

}


// Sets the mesh to an initial value, determined by the MMS scheme
void initial_value(const int n, const double dx, const double length, double *
    restrict u) {

  double y = dx;
  for (int j = 0; j < n; ++j) {
    double x = dx; // Physical x position
    for (int i = 0; i < n; ++i) {
      u[i+j*n] = sin(PI * x / length) * sin(PI * y / length);
      x += dx;
    }
    y += dx; // Physical y position
  }
}


// Zero the array u
void zero(const int n, double * restrict u) {
  for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
      u[i+j*n] = 0.0;
    }
  }
}


```

```
// Compute the next timestep, given the current timestep
void solve(const int n, const double alpha, const double dx, const double dt, const double * restrict u, double * restrict u_tmp) {

  // Finite difference constant multiplier
  const double r = alpha * dt / (dx * dx);
  const double r2 = 1.0 - 4.0*r;

  // Loop over the nxn grid
  #pragma omp target teams loop collapse(2) map(tofrom: u[0:n*n], u_tmp[0:n*n])
  for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {

      // Update the 5-point stencil, using boundary conditions on the edges of the domain.
      // Boundaries are zero because the MMS solution is zero there.
      u_tmp[i+j*n] = r2 * u[i+j*n] +
      r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
      r * ((i > 0)  ? u[i-1+j*n] : 0.0) +
      r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
      r * ((j > 0)  ? u[i+(j-1)*n] : 0.0);
    }
  }
}


// True answer given by the manufactured solution
double solution(const double t, const double x, const double y, const double alpha, const double length) {

  return exp(-2.0*alpha*PI*PI*t/(length*length)) * sin(PI*x/length) * sin(PI*y/length);

}


// Computes the L2-norm of the computed grid and the MMS known solution
// The known solution is the same as the boundary function.
double l2norm(const int n, const double * restrict u, const int nsteps, const double dt, const double alpha, const double dx, const double length) {

  // Final (real) time simulated
  double time = dt * (double)nsteps;

  // L2-norm error
  double l2norm = 0.0;

  // Loop over the grid and compute difference of computed and known solutions as an L2-norm
  double y = dx;
  for (int j = 0; j < n; ++j) {
    double x = dx;
    for (int i = 0; i < n; ++i) {
      double answer = solution(time, x, y, alpha, length);
      l2norm += (u[i+j*n] - answer) * (u[i+j*n] - answer);

      x += dx;
    }
```

```
275     y += dx;
276   }
277 
278   return sqrt(l2norm);
279 
280 }
```

Código 2: tarefa20.c